信息学竞赛培训 · C++ 程序设计

LESSON 05 / DP STAGE ONE

0/1 Knapsack — From Two Dimensions to Rolled Array

01 背包入门

状态设计 · 二维 DP · 一维滚动数组 · 倒序枚举

> 背包问题的关键只有一句话：对每一件物品，决定「选它」或「不选它」。把这个决策按顺序一个一个做下去，所有方案就在一张表里被完整枚举了。

01

授课班级
中预 2026 级 B 班

主讲
信息教研组

时间
2026 年 10 月

配套 3 道洛谷例题精讲

学习目标与课堂安排

第 5 课 · 01 背包专题

本课要解决的问题

本课共 22 页。前 11 页把 01 背包的「为什么」讲透，后 11 页落到代码与常见坑，最后给出三道洛谷例题。

1 看懂「选或不选」

把带限制的最优化问题，翻译成「对每件物品做一次二选一决策」的模型，这是所有背包问题的入口。

2 写出二维 DP

定义 dp[i][j] 的含义，推导出状态转移方程，并说清边界为什么这么取。

3 压成一维数组

理解滚动数组为什么能省掉一维，以及内层循环为什么必须倒序枚举。

4 避开三类高频错误

倒序写成正序、下标越界、边界初始化——这三个坑占新手错误的绝大多数。

本课路线图

第 5 课 · 01 背包专题

从问题到代码的四步

沿着下面这条主线走：先承认暴力不可行，再引入「状态 + 决策」，最后把它写成两重循环。

① 问题模型	→	② 暴力为何不行	→	③ 状态与转移	→	④ 空间优化
容量 + 物品 + 选或不选		2 的 n 次方种方案，不可枚举		dp 表 + 一行转移方程		二维压一维 + 倒序枚举

一句话先记住

01 背包 = 有容量上限、每件物品最多选一次的最优化问题。

解题口诀：定义「前 i 件、容量 j」的最优值 → 对第 i 件决定选或不选 → 转移方程取两者较大值 → 用循环把整张表填满。

下面三页先解决「为什么不能贪心」，再正式进入状态设计。

什么是 01 背包问题

先看最朴素的说法：有一个容量有限的背包，面前摆着一堆物品，每件物品有「重量」和「价值」。挑一些装进去，在总重量不超限的前提下，让总价值最大。

生活里的三个例子

· 采药：时间是容量，草药是物品，挖草耗费时间、卖钱得价值

· 装行李：箱子承重是容量，每件行李有重量和必要性

· 购物预算：预算是容量，每件商品有价格和喜爱度

为什么叫「01」

每件物品只有两种归宿：放进背包（记为 1），或者不放进（记为 0）。

不能放一半，也不能放两次 —— 这就是「0 / 1」两个数字的全部含义。

换个说法：每件物品最多被选一次。

形式化描述（把上面的话翻译成数学）

给定 n 件物品与背包容量 C。第 i 件物品重量为 w[i]，价值为 v[i]。

要求选出一个子集 S，使得「所选物品重量之和 ≤ C」，并且「所选物品价值之和」最大。

每件物品只有「选」或「不选」两种可能，所以一共有 2 的 n 次方种候选方案 —— 这就是下一页要面对的困难。

为什么贪心算法不行

第一反应往往是这样：把物品按「性价比」（价值 ÷ 重量）从高到低排，依次往背包里塞，塞不下就跳过。

这个策略看起来很聪明，但它会出错。下面用一个具体反例说明。

反例数据

背包容量 C = 10。

物品 A：重量 6，价值 12，性价比 2.0

物品 B：重量 5，价值 9， 性价比 1.8

物品 C：重量 5，价值 9， 性价比 1.8

贪心的选择与后果

贪心先拿 A（性价比最高），用掉 6 容量，剩 4。

B、C 各需 5，都装不进 —— 于是贪心得到总价值 12。

但真正的最优解是「B + C」：重量 5 + 5 = 10，价值 9 + 9 = 18。

结论

贪心只看眼前的局部最优，而背包问题的选择之间会互相影响：先拿走 A，就等于放弃了 B + C 的组合。

要拿到真正的最优解，必须把所有「选 / 不选」的组合都考虑一遍 —— 但 2 的 n 次方种枚举又不可行。

于是我们需要一种「不重复、不遗漏地压缩状态」的办法，这就是动态规划。

把所有决策排成一排

换一个视角看问题：不要想着「一次挑出最优子集」，而是按物品编号 1 到 n，一件一件地做决定。

每做完一次决定，我们就回答一个问题：到目前为止，在给定的容量限制下，能达到的最好价值是多少？

第 1 件

决定第 1 件物品选或不选，记录当前容量下能达到的最优价值。

第 2 件

在第 1 件的基础上，再决定第 2 件的去留，价值可以累加。

第 3 件

同理继续。每一步都只依赖「前面已决定完的物品」的结果。

… 直到第 n 件

全部决定完，容量内能取到的最大价值就是答案。

关键洞察：决策是有顺序的，且后一步只依赖前一步的结果 —— 这正是「动态规划」适用的信号。

第一步：设计状态

动态规划的第一件事，是给「子问题」起个名字。背包问题里有两条线索会不断变化：处理到第几件物品，以及还剩多少容量。

状态定义（务必逐字读三遍）

dp[i][j] 表示：只考虑前 i 件物品，在容量正好不超过 j 的前提下，能取得的最大总价值。

其中 i 的取值范围是 0 到 n，j 的取值范围是 0 到 C。

两个下标分别管什么

i（行）：已经做完决定的物品数量。

i = 0 表示一件都还没考虑。

j（列）：当前可用的容量上限。

j = 0 表示一点容量都不剩。

dp[i][j] 这个格子，就是这两个问题的交叉答案。

为什么这样设计就够了

任意一个「前 i 件 + 容量 j」的子问题，最优解都由更小的子问题拼出来。

因此只要把 i、j 的所有组合都算一遍，答案就藏在 dp[n][C] 里。

这就是「用空间换时间」：用一张表，把 2 的 n 次方种枚举压缩成 n × C 个格子。

第二步：状态转移方程

现在盯着第 i 件物品，它只有两种命运。把这两种命运分别算出来的价值比一比，取大的那个，就是 dp[i][j]。

命运一：不选第 i 件

既然不选它，那就等于问题退化成「只有前 i - 1 件物品、容量还是 j」。

对应值：dp[i - 1][j]

命运二：选第 i 件

前提是 j ≥ w[i]。选了它，容量要扣掉 w[i]，价值加上 v[i]。

对应值：dp[i - 1][j - w[i]] + v[i]

状态转移方程

若 j 小于 w[i]（装不下）：dp[i][j] = dp[i - 1][j]

若 j 大于等于 w[i]（装得下）：dp[i][j] = max(dp[i-1][j] , dp[i-1][j - w[i]] + v[i])

第一项是「不选」，第二项是「选」。取较大值，就完成了一次决策。

第三步：边界与初始化

递推要有起点。有两类边界必须一开始就填好，否则整张表都会算错。

第 0 行：一件物品都没有

i = 0 时，没有任何物品可选，所以不管容量多大，能拿到的价值都是 0。

dp[0][j] = 0，对所有 j 成立。

第 0 列：容量为 0

j = 0 时，背包一点容量都没有，任何物品都装不下，价值同样是 0。

dp[i][0] = 0，对所有 i 成立。

注意：其它格子不要预先赋值

除了这两条边界，其余格子会在两重循环中被依次算出，不需要（也不应该）手工初始化。

用全局数组时天然全为 0，正好满足边界条件，这是 C++ 里最常见的写法。

如果题目要求「必须恰好装满」，边界就要改成 dp[0][0] = 0、其余为负无穷，本课最后一节会讲。

手算一遍：把表填满

用一个小数据走一遍。容量 C = 5，三件物品：(重量 2, 价值 3)、(重量 3, 价值 4)、(重量 4, 价值 5)。

i=0（无物品）	0	0	0	0	0	0
i=1（加物品1）	0	0	3	3	3	3
i=2（加物品2）	0	0	3	4	4	7
i=3（加物品3）	0	0	3	4	5	7

怎么读这张表

看 i=2、j=5 这一格：装不下物品 2（需 3，但可以）→ 比较「不选」dp[1][5]=3 与「选」dp[1][2]+4=3+4=7，取 7。

答案就在右下角 dp[3][5] = 7，对应方案是「物品 1 + 物品 2」，重量 2 + 3 = 5，价值 3 + 4 = 7。

二维 DP 完整代码


```cpp
int dp[105][1005];          // dp[i][j]：前 i 件、容量 j 的最大价值
int w[105], v[105];         // 每件物品的重量与价值

int main() {
    int C, n;
    cin >> C >> n;
    for (int i = 1; i <= n; i++)
        cin >> w[i] >> v[i];

    for (int i = 1; i <= n; i++) {      // 逐件物品做决定
        for (int j = 0; j <= C; j++) {  // 枚举每一个容量
            if (j < w[i])               // 装不下：只能不选
                dp[i][j] = dp[i-1][j];
            else                        // 装得下：选与不选取较大
                dp[i][j] = max(dp[i-1][j], dp[i-1][j-w[i]] + v[i]);
        }
    }
    cout << dp[n][C] << endl;           // 答案在右下角
    return 0;
}
```

逐件推进

外层 i 从 1 到 n，保证算 dp[i] 时 dp[i-1] 已经全部算好。

容量枚举

内层 j 从小到大，二维写法不用考虑顺序，正序倒序都正确。

答案位置

dp[n][C] 就是最终答案，不必再遍历求最大值。

空间代价

二维数组需要 n × C 个格子。当 C 达到 30000 时，100 × 30000 的 int

第四步：把二维压成一维

回顾转移方程，算 dp[i][j] 时只用到了 dp[i-1] 这一整行。既然第 i-2 行以及更早的行再也不需要，那还有必要把它们留着吗?

关键观察：每一行只依赖上一行

dp[i][j] = max(dp[i-1][j] , dp[i-1][j - w[i]] + v[i])

式子右边出现的全是 i-1 行，一次都没有用到 i-2、i-3。

也就是说：只要保留「上一行」的完整信息，就能算出「这一行」。

做法：用一维数组原地覆盖

把二维数组 dp[i][j] 压成 dp[j]，它的含义变成：「当前容量 j 下能达到的最大价值」。

处理第 i 件物品时，我们直接在 dp 数组上做修改，让它从「前 i-1 件的结果」变成「前 i 件的结果」。

这样空间复杂度从 O(n × C) 降到 O(C) —— 当容量很大时，这是能不能过题的关键。

关键细节：内层循环必须倒序

压成一维后，转移方程写成：dp[j] = max(dp[j] , dp[j - w[i]] + v[i])。

但这时 j 的枚举方向变得致命 —— 只能从大到小，不能从小到大。

如果正序枚举（错误做法）

j 从 0 到 C 递增。当算到 dp[j] 时，需要读 dp[j - w[i]]。

而 j - w[i] 比 j 小，在正序循环里它「已经在本轮被更新过了」。

于是 dp[j - w[i]] 里装的可能已经是「选过第 i 件」的结果 —— 相当于第 i 件被重复使用了。

结果：01 背包被悄悄写成了完全背包（每件可无限取）。

倒序枚举（正确做法）

j 从 C 递减到 w[i]。因为 j - w[i] 比 j 小，倒序时它「还没被本轮碰过」，仍是第 i-1 件的旧值。

这正好对应二维写法里的 dp[i-1][j - w[i]]，语义完全一致。

一句话记忆：一维 01 背包，容量必须倒着扫。

正序 vs 倒序：一张图看懂

以「物品重量 w = 2，价值 v = 3，容量 C = 5」为例，看同一个物品在两轮扫描中会造成什么差别。

正序 j = 0 → 5（会把物品用两次）

j=0,1：装不下，不变。

j=2：dp[2] = max(0, dp[0]+3) = 3

j=4：dp[4] = max(0, dp[2]+3) = 3+3 = 6 ← dp[2] 已是本轮新值

结果 dp[4] = 6，相当于把同一件物品拿了两次。

倒序 j = 5 → 0（正确）

j=5：dp[5] = max(0, dp[3]+3) = 3 ← dp[3] 还是旧值

j=4：dp[4] = max(0, dp[2]+3) = 3 ← dp[2] 也还是旧值

j=3：dp[3] = max(0, dp[1]+3) = 3

j=2：dp[2] = max(0, dp[0]+3) = 3

结果每个容量最多被这件物品加一次。

牢记这一点

正序枚举会让 dp[j - w[i]] 读到本轮刚更新的值，等价于允许重复选，那是完全背包。

01 背包要求「每件最多一次」，所以内层必须倒序 —— 这是最容易写错、也是调试时最难发现的一个坑。

一维滚动数组完整代码


```cpp
int dp[1005];                // dp[j]：容量 j 下能取得的最大价值
int w[105], v[105];

int main() {
    int C, n;
    cin >> C >> n;
    for (int i = 1; i <= n; i++)
        cin >> w[i] >> v[i];

    for (int i = 1; i <= n; i++) {          // 逐件物品
        for (int j = C; j >= w[i]; j--) {   // 容量倒序枚举！
            dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
        }
    }
    cout << dp[C] << endl;                  // 答案在 dp[C]
    return 0;
}
```

循环上界起手

内层从 C 开始，止于 w[i] —— 比 w[i] 还小的容量根本装不下，不必算。

倒序不可省

j-- 是本题灵魂。写成 j++ 就变成了完全背包，样例常常还能过，但会 WA 在数据上。

答案在 dp[C]

一维数组不需要再找最大值，dp[C] 就是全容量下的最优解。

内存对比

二维需要 n × C 个 int；一维只要 C 个。以 C = 30000、n = 100 为例，

用样例验证一遍

拿 P2871 的样例走一遍（容量 C = 6，四件物品）。下表列出每处理完一件物品后，dp[0] 到 dp[6] 的值。

初始（无物品）	0	0	0	0	0	0	0
物品1 (1,4)	0	4	4	4	4	4	4
物品2 (2,6)	0	4	6	10	10	10	10
物品3 (3,12)	0	4	6	12	16	18	22
物品4 (2,7)	0	4	7	12	16	19	23

最终答案 dp[6] = 23

对应方案：物品 1（重 1，值 4）+ 物品 3（重 3，值 12）+ 物品 4（重 2，值 7），总重 6，总价值 23。

注意看「物品 3」那一行，dp[6] 从 10 跳到 22，说明每件物品最多只被加了一次。

复杂度分析与数据范围适配

同样的算法，用在不同的数据范围上，写法可能要微调。下面三题恰好覆盖了三种典型规模。

P1048 采药	≤ 1000	≤ 100	约 10 万	二维 / 一维均可
P1060 开心的金明	< 30000	< 25	约 75 万	建议一维，省内存
P2871 Charm Bracelet	≤ 12880	≤ 3400	约 4400 万	必须一维，二维易超内存

两点工程提醒

第一，数组要开到「容量上限 + 1」并留出余量，比如容量上限 12880 就开 13000，避免越界写坏内存。

第二，n × C 很大时（P2871 约 4400 万次操作），要确保内层循环体足够简洁，别在里面放函数调用或字符串操作。

变式：要求「恰好装满」

有一类题会额外要求「背包必须被正好装满」。这时转移方程一个字都不用改，只要改初始化。

不要求装满（本课三道例题都属此类）

数组全部初始化为 0。

含义：容量用不完是允许的，「什么都没装」也算一种合法方案，价值为 0。

要求恰好装满

dp[0] = 0，其余 dp[j] = 负无穷（通常取 -0x3f3f3f3f）。

含义：只有容量 0 是「合法的已装满状态」；其它容量在没装东西时属于「无法达到」。

最后如果 dp[C] 仍是负无穷，说明恰好装满的方案不存在；否则 dp[C] 就是答案。

三类高频错误

下面三个坑几乎每个初学 01 背包的人都踩过。做题时对照检查一遍，能省下大量调试时间。

01

内层写成正序

j++ 会让物品被重复选取，01 背包退化成完全背包。样例往往恰好能过，但隐藏数据必错。口诀：容量倒着扫。

02

数组开小越界

dp 的下标上界是容量 C，数组要开到 C+1 以上。P2871 容量 12880，至少要 int dp[12881]。

03

边界误初始化为负无穷

只有「恰好装满」类题目才需要设负无穷。普通题设了负无穷，会把大量可行方案误判为不可达。

本课小结

把整节课压缩成下面五句话。合上课件，能复述出来，就说明真的懂了。

01 状态

dp[j] 表示容量 j 下能取得的最大价值（二维版是 dp[i][j]）。

02 转移

dp[j] = max(dp[j] , dp[j - w[i]] + v[i])，前者不选，后者选。

03 方向

内层容量必须倒序枚举，否则物品会被重复使用。

04 边界

普通题全初始化为 0；要求恰好装满时，dp[0]=0、其余为负无穷。

05 答案

一维写法答案即 dp[C]，二维写法答案在 dp[n][C]。

课堂自测

第 5 课 · 01 背包专题

检验掌握程度

不用敲代码，先在纸上回答下面五问。能全部答对，说明 01 背包的骨架已经立住了。

Q1

为什么 01 背包不能按性价比贪心?

答案要点：需要一个具体的反例来说明（第 5 页那个例子即可）。

Q2

dp[i][j] 的准确含义是什么?

答案要点：要点：前 i 件物品、容量不超过 j、最大总价值。三个限定缺一不可。

Q3

转移方程里的两项分别代表什么?

答案要点：dp[i-1][j] 是不选第 i 件；dp[i-1][j-w[i]]+v[i] 是选第 i 件。

Q4

压成一维后，内层为什么必须倒序?

答案要点：因为正序会让 dp[j-w[i]] 读到本轮新值，等于允许重复选取。

Q5

什么情况下要把 dp 初始化为负无穷?

答案要点：当题目要求「恰好装满」时；普通题保持全 0 即可。

把决策一件一件地做下去

状态定义清楚了，转移方程就只是一次二选一

课后任务：把三道例题各自手敲一遍 · 二维与一维各写一次

下一节 · 三道例题的解题思路与代码精讲